Name → Sudhanshu UID → 23BCS12388 Class → KRG-1_A

Q.) Given 3 integers n, a & b. Return the nth magical no.

Ans →

```
int Solution(int n, int a, int b){
    int gc = gcd(a,b);
long ~~int~~ common = a*1LL*b/gc;
    ~~int low = min(a,b)~~
    long long low = min(a,b), high = low*n;
    ~~int~~
    while (low <= high){
        long long ~~total~~ mid = (low+high)/2;
        long long total = mid/a + mid/b - mid/common
        if(total == n){
            return max(a*(mid/a), b*(mid/b))%1000000000
        else if(total<n){
            low = mid+1;
        else {
            high = mid-1;
        }
    }
    return (min(a,b) * n)%(1e9+7);
}
```

Time Complexity ⇒ $O(\log(n * \min(a,b)))$

Better Code =>

```
int Solution (int a, int b, int n){
    int gc = gcd (a, b);
    long long common = a * 1LL * b / gc;
    int low = 1, high = n, small = min(a, b), big = max
                                                (a, b);
    while (low <= high) {
        long long mid = low + (high - low)/2;
        long long total = mid + (small * mid / big)
                                - (small * mid / common);
        if (total == n) return (mid * small)%(1e9+7);
        else if (total < n) low = mid + 1;
        else high = mid - 1;
    }
    ~~while~~.
    int low = 1, high = small * n / big;
    while (low <= high) {
        long long mid = low + (high - low)/2;
        longlong total = mid + (big * mid / small)
                                - (big * mid / common);
        if (total == n) return (mid * big)%(1e9+7);
        else if (total < n) low = mid + 1;
        else high = mid - 1;
    }
    return 0;                    // Time Complexity -> O(log n)
}
```